객체지향 프로그래밍 고급

10강 : 추상클래스

순수가상함수

- 가상함수는 재정의가 가능한 함수이다.(반드시 재정의할 필요는 없다.)
- **순수가상함수는 파생 클래스에서 반드시 재정의해야 하는 멤버 함수를 의미한다.**
- **반드시 사용되어야 하는 멤버함수를 순수가상함수로 선언하면, 이 클래스로부터 파생된 모든 클래스에서는 이 가상함수를 반드시 재정의해야 한다.**

```
virtual 멤버함수의 원형=0;
virtual void Cry()=0;
```

- 순수가상함수는 함수의 동작을 정의하는 본체를 갖고 있지 않기 때문에 파생 클래스에서 재정의하지 않으면 사용할 수 없다.

순수가상함수의 예

```
class Animal
{
public:
    Animal() {}
    virtual void Cry()=0;
};
 class Dog : public Animal
{
public:
    virtual void Cry() { cout << "멍
멍!!" << endl; }
};
 class Cat : public Animal
{
public:
    virtual void Cry() { cout << "야
옹!!" << endl; }
};
```

```
int main()
{
    Dog mydog;
    mydog.Cry();
    Cat mycat;
    mycat.Cry();
    return 0;
}
```

추상클래스

- 하나 이상의 순수가상함수를 포함하는 클래스를 추상클래스(abstract class)라고 한다.**(객체생성이 불가능한, 완전하지 않은 클래스)**
- **추상클래스는 동작이 정의되지 않은 순수가상함수를 포함하므로, 객체를 생성할 수 없다.**
- 추상클래스는 다형성을 가진 함수의 집합을 정의할 수 있게 한다.
- 추상클래스의 장점
  - 잘못된 객체의 생성을 막을 수 있다.
  - 자식 클래스에 정의된 함수가 호출되게끔 돕는 의미만 있고 실제로 실행이 되는 함수가 아닐 경우, 이를 명확히 명시할 수 있다.

추상클래스의 사용(1)

```
class Animal
{
public:
    Animal() {}
    virtual void Cry() {};
};
 class Dog : public Animal
{
public:
    virtual void Cry() { cout << "멍
멍!!" << endl; }
};
 class Cat : public Animal
{
public:
    virtual void Cry() { cout << "야
옹!!" << endl; }
};
```

```
int main()
{
    Animal myAnimal;
    . . .
}
```

Animal myAnimal;
→ 컴파일 오류는 발생하지 않으나 의미 없는 객체가 됨

추상클래스의 사용(2)

```
class Animal
{
public:
    Animal() {}
    virtual void Cry() = 0;
};
 class Dog : public Animal
{
public:
    virtual void Cry() { cout << "멍
멍!!" << endl; }
};
 class Cat : public Animal
{
public:
    virtual void Cry() { cout << "야
옹!!" << endl; }
};
```

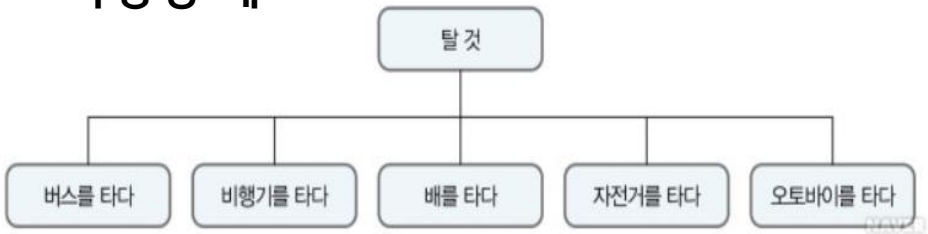
```
int main()
{
    Animal myAnimal;
    . . .
}
```

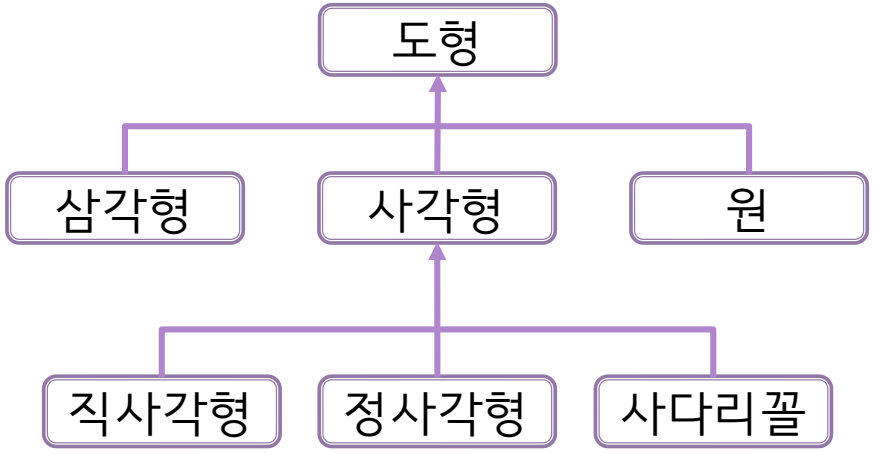
Animal myAnimal;
→ 컴파일 오류 발생

다형성(Polymorphism)

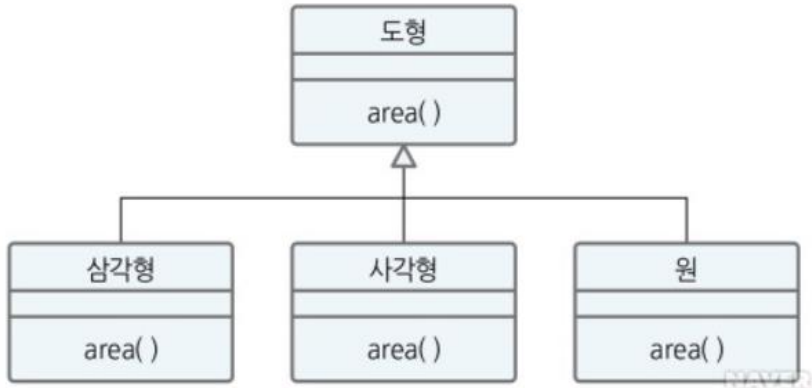
: 하나 이상의 형태

다형성 예





함수 오버로딩





[예제] 추상클래스

Vehicle 클래스와 이로부터 파생된 Car 와 Plane클래스가 있다. Vehicle은 추상클래스이며, 각각의 멤버변수와 멤버함수는 아래와 같다.
Vehicle 클래스의 객체포인터배열을 이용하여 아래 데이터를 출력하시오.

Vehicle class
- int speed(속도)
- void Show()

Car class
- 자동차 번호(int num)
- 가솔린 양(double gas)
- void Show()

Plane class
- 편명(int flight)
- void Show()

자동차
번호 : 8604, 가솔린양: 25.5, 속도: 80);
비행기
편명 : 940, 속도 : 500

```
int main()
{
	Vehicle *pv[2];
	Car c(8604, 25.5, 80);
	pv[0] = &c;
	Plane pl(940, 500);
	pv[1] = &pl;
```

가상 소멸자

First와 Second클래스의 생성자가 언제 호출되는가?

```
class First
{
private:
	char * strOne;
public:
	First(char * str)
	{
		strOne = new char[strlen(str) + 1];
	}
	~First()
	{
		cout << "~First" << endl;
		delete[]strOne;
	}
};
```

```
class Second : public First
{
private:
	char * strTwo;
public:
	Second(char * str1, char * str2) : First(str1)
	{
		strTwo = new char[strlen(str2) + 1];
	}
	~Second()
	{
		cout << "~Second()" << endl;
		delete[]strTwo;
	}
};

int main()
{
	First * ptr = new Second("Simple", "Complex");
	delete ptr;
	return 0;
}
```

가상 소멸자

이 프로그램이 종료될 때의 문제점은 무엇인가?

```
class First
{
private:
	char * strOne;
public:
	First(char * str)
	{
		strOne = new char[strlen(str) + 1];
	}
	~First()
	{
		cout << "~First" << endl;
		delete[]strOne;
	}
};
```

```
class Second : public First
{
private:
	char * strTwo;
public:
	Second(char * str1, char * str2) : First(str1)
	{
		strTwo = new char[strlen(str2) + 1];
	}
	~Second()
	{
		cout << "~Second()" << endl;
		delete[]strTwo;
	}
};

int main()
{
	First * ptr = new Second("Simple", "Complex");
	delete ptr;
	return 0;
}
```

가상 소멸자

객체의 소멸을 First형 포인터로 명령하면(delete ptr;), First 클래스의 소멸자만 호출

→ **메모리 누수(leak) 발생** : Second 클래스의 소멸자가 호출되지 않아 동적할당 된 메모리가 해제되지 않는다.

→ **객체의 소멸과정에서는 delete 연산자에 사용된 포인터 변수의 자료형에 상관없이 모든 소멸자가 호출되어야 한다.**

가상 소멸자

- 상속의 계층구조상 맨 위에 존재하는 부모클래스의 소멸자만 virtual로 선언하면, 이를 상속하는 자식클래스의 소멸자들도 모두 가상소멸자로 선언이 된다.
- 가상 소멸자가 호출되면, 자식클래스의 소멸자가 대신 호출되면서 기초클래스의 소멸자가 순차적으로 호출된다.

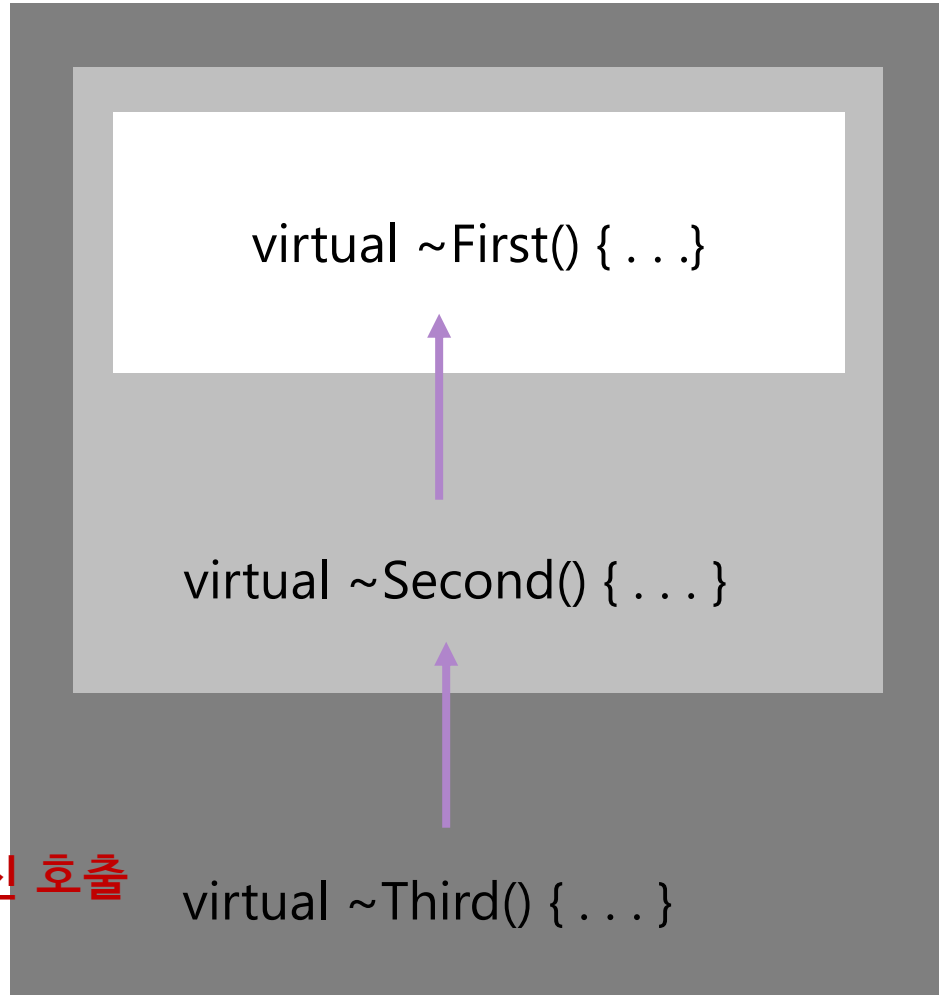
```
virtual ~First()
{
    cout << "~First" << endl;
    delete [] strOne;
}
```

가상 소멸자

```
class First
{
. . .
public:
    virtual ~First() { . . }
};
class Second : public First
{
. . .
public:
    virtual ~Second() { . . }
};
class Third : public Second
. . .
public:
    virtual ~Third() { . . }
};
```

```
int main()
{
    First * ptr = new Third();
    delete ptr;
}
```

가상 소멸자의 실행 과정



(1)객체소멸과정에서 호출
delete ptr;

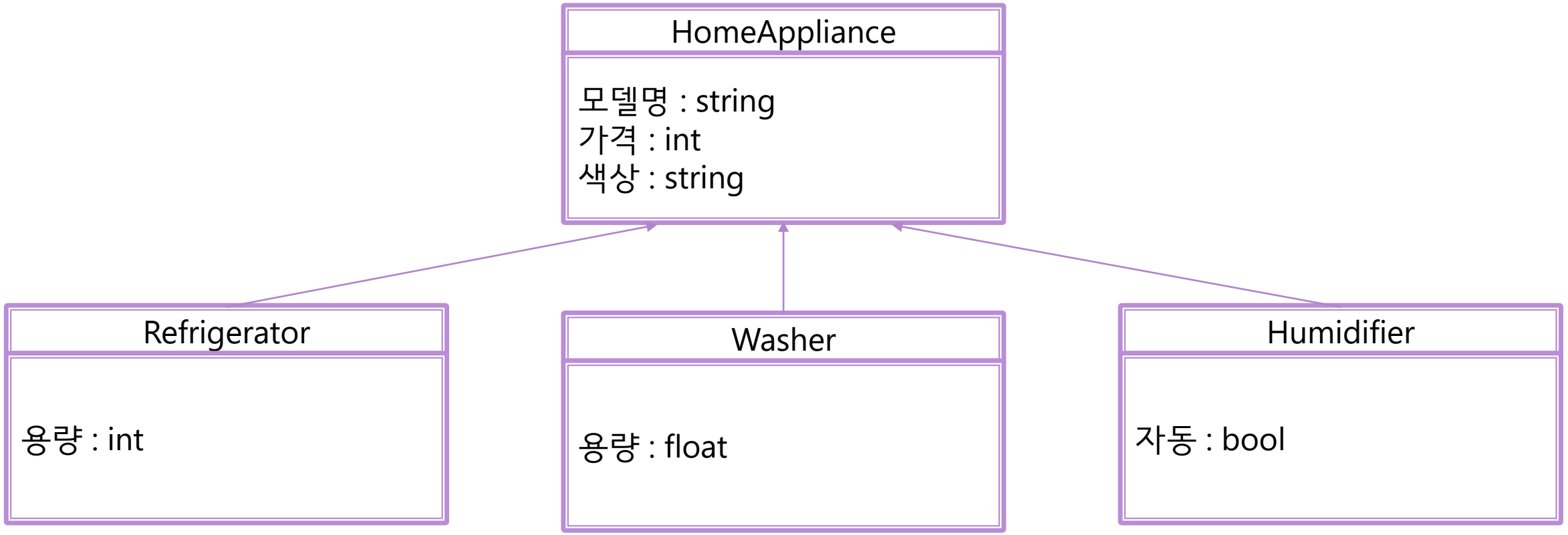
(2)virtual소멸자이므로 대신 호출
virtual ~First() { . . }

(3)~Third() 실행후에 이어서 실행

(4)~Second() 실행후에 이어서 실행

[예제]

냉장고, 세탁기, 가습기 등과 같은 전자제품을 판매하기 위해서 상품정보(모델명, 브랜드, 가격)를 저장하기 위한 클래스를 만들려고 한다. 이들 클래스들의 공통분모로 사용되는 정보는 부모 클래스에 정의해 두고 이를 상속받는 자식 클래스로 설계하시오.



friend함수

- private으로 선언된 멤버변수는 선언된 클래스 외부에서는 접근할 수 없다.
- friend 키워드를 붙인 멤버함수는 어떤 클래스에도 속하지 않는 독립된 일반 외부 함수로 클래스의 모든 멤버변수에 접근할 수 있는 전역함수이다.
- 일반함수나 다른 클래스에 있는 함수를 friend함수로 사용할 수 있다.
- friend함수 용도

  - 연산자를 오버로드하여 재정의할 때 사용
  - 두 개 이상의 클래스에서 서로 연관되는 멤버 변수들을 포함하고 있을 때 각 객체의 상태를 파악하기 위함

①일반함수를 friend함수로 사용

```
class Dog
{
	int age;
	int weight;
	public:
	Dog(int a, int w);
	friend void DataShow(Dog &obj);
};

Dog::Dog(int a, int w)
{
	age = a;
	weight = w;
}
```

```
void DataShow(Dog &obj)
{
	cout << "나이:" << obj.age << endl;
	cout << "몸무게:" << obj.weight << endl;
}

int main()
{
	Dog happy = Dog(3, 4);
	DataShow(happy);
	return 0;
}
```

②다른 클래스에 있는 함수를 friend함수로 사용

```
class Dog;
class Cat
{
	int age;
	int weight;
public:
	Cat(int a, int w) { age = a; weight = w; }
	void DataShow(Dog &obj);
};

void Cat::DataShow(Dog &obj)
{
	cout << "나이:" << obj.age << endl;
	cout << "몸무게:" << obj.weight << endl;
}
```

```
class Dog
{
	friend class Cat;
	int age;
	int weight;
	public:
	Dog(int a, int w) { age = a, weight = w; }
	friend void Cat::DataShow(Dog &obj);
};

int main()
{
	Dog happy = Dog(3, 4);
	Cat cat = Cat(5, 7);
	cat.DataShow(happy);
	return 0;
}
```

다중상속

다중상속의 예

```
class BaseOne
{
public:
	void SimpleFuncOne() { cout << "BaseOne\n"; }
};
class BaseTwo
{
public:
	void SimpleFuncTwo() { cout << "BaseTwo\n"; }
};
class MultiChild : public BaseOne, public BaseTwo
{
public:
	void ComplexFunc()
	{
		SimpleFuncOne();
		SimpleFuncTwo();
	}
};
```

```
int main()
{
	MultiChild drv;
	drv.ComplexFunc();

	return 0;
}
```

다중상속의 모호성

- 부모 클래스들의 멤버함수 이름이 동일한 경우 어느 클래스의 멤버에 접근해야 할지 애매하다.

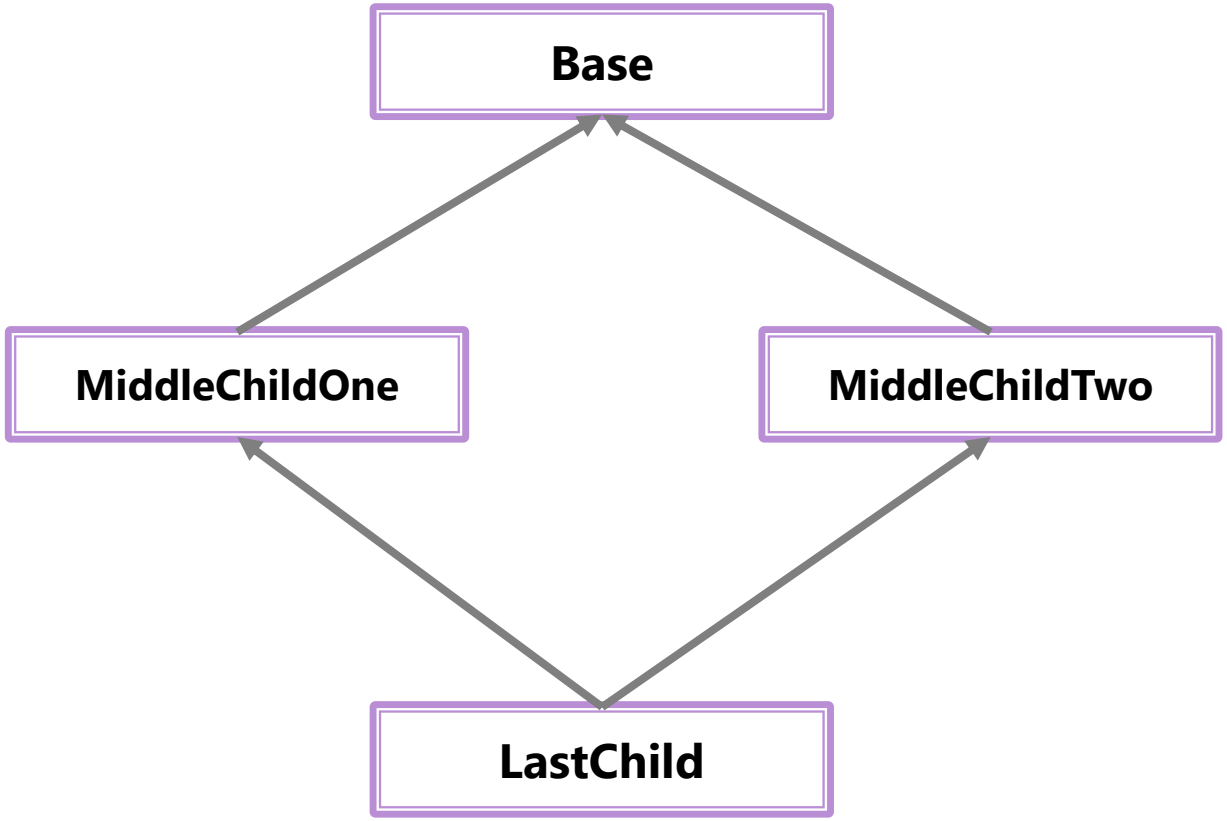
-> 멤버함수이름 앞에 클래스이름을 붙여줘야 한다.

```
class BaseOne
{
public:
    void SimpleFunc() { cout << "BaseOne" << endl; }
};

Class BaseTwo
{
Public:
    void SimpleFunc() { cout << "BaseTwo" << endl; }
};
```

```
class MultiChild : public BaseOne, public BaseTwo
{
public:
    void ComplexFunc()
    {
            BaseOne::SimpleFunc();
            BaseTwo::SimpleFunc();
    }
};
```

다중상속의 문제점 : Diamond's Problem



Base생성자가 몇 개 생기는가?

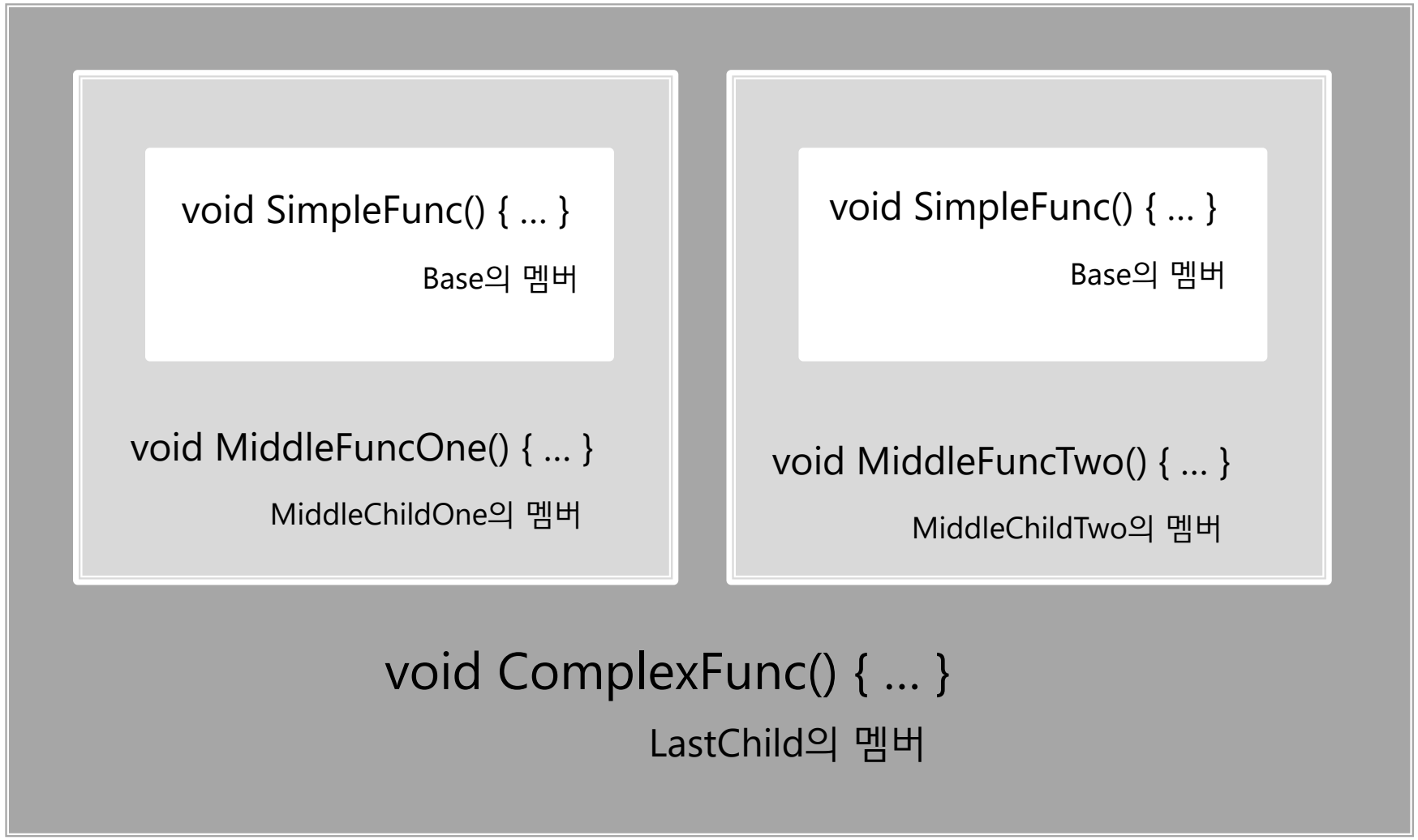
```
Base() { cout << "Base 생성자\n"; }
void SimpleFunc() { cout << "BaseOne\n"; }

MiddleChildTwo() {
        cout << "MiddleDerivedTwo 생성자\n"; }
void MiddleFuncTwo() {
        SimpleFunc();
        cout << "MiddleDerivedTwo\n"; }

LastChild() {
        cout << "LastDerived생성자\n"; }
void ComplexFunc() {
        SimpleFunc();
        MiddleFuncOne();
        MiddleFuncTwo();
}
```

```
class Base
{
public:
	Base() {
		cout << "Base 생성자\n";
	}
	void SimpleFunc() {
		cout << "BaseOne\n";
	}
};
class MiddleChildOne : public Base
{
public:
	MiddleChildOne() {
		cout << "MiddleDerivedOne생성자\n";
	}
	void MiddleFuncOne()
	{
		SimpleFunc();
		cout << "MiddleDerivedOne\n" << endl;
	}
};
```

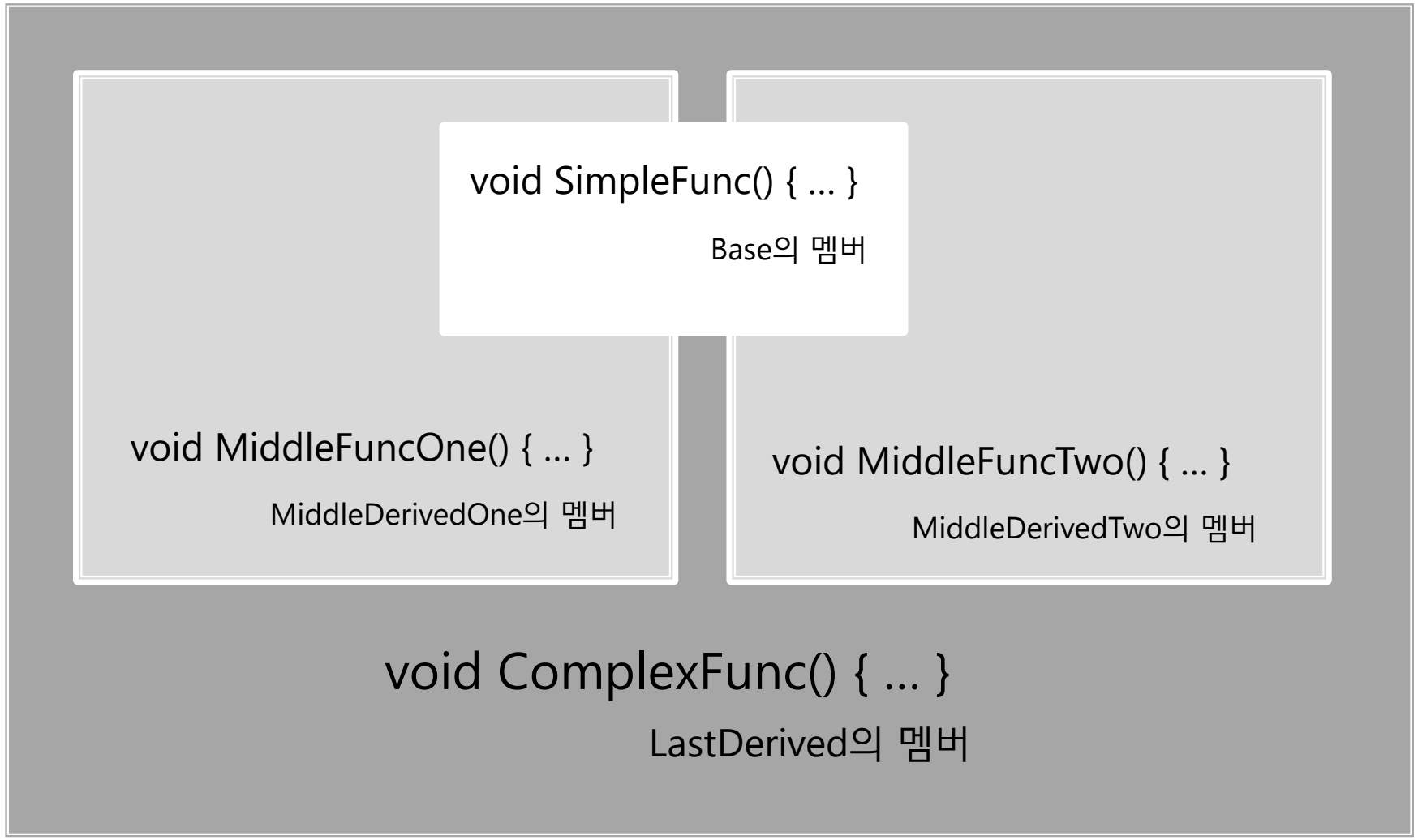
```
class LastChild : public MiddleChildOne, public MiddleChildTwo
{
public:
	LastChild() {
		cout << "LastDerived생성자\n";
	}
	void ComplexFunc()
	{
		MiddleFuncOne();
		MiddleFuncTwo();
		Base::SimpleFunc();
	}
};
int main()
{
	cout << "객체생성전\n";
	LastChild obj;
	obj.ComplexFunc();
	return 0;
}
```


void SimpleFunc() { … }
Base의 멤버
void MiddleFuncOne() { … }
MiddleChildOne의 멤버
void SimpleFunc() { … }
Base의 멤버
void MiddleFuncTwo() { … }
MiddleChildTwo의 멤버
void ComplexFunc() { … }
LastChild의 멤버

가상상속 : 다이아몬드 문제의 해결 방법

```
class MiddleChildOne : virtual public Base

class MiddleChildTwo : virtual public Base
```


void SimpleFunc() { … }
Base의 멤버
void MiddleFuncOne() { … }
MiddleDerivedOne의 멤버
void MiddleFuncTwo() { … }
MiddleDerivedTwo의 멤버
void ComplexFunc() { … }
LastDerived의 멤버